

# C++ Coroutine TS Issues

|            |                                                                             |
|------------|-----------------------------------------------------------------------------|
| Doc. no.   | P0664R5                                                                     |
| Revises    | P0664R4                                                                     |
| Date:      | Revised 2018-06-24                                                          |
| Project:   | Programming Language C++                                                    |
| Reference: | ISO/IEC TS 22277, C++ Extensions for Coroutines                             |
| Audience:  | CWG                                                                         |
| Reply to:  | Gor Nishanov < <a href="mailto:gorn@microsoft.com">gorn@microsoft.com</a> > |

## Introduction

All proposed resolutions wording is relative to [N4736](#).

Previous issue list is [P0664R4](#).

## Table of content

- CWG issues: [24](#) [25](#) [26](#) [27](#) [28](#)
- Adopted at Rapperswil 2018 issues: [29](#)
- Not ready issues: [30](#) [31](#) [32](#) [33](#)

## CWG Issues

---

### [24](#). The specification of initial suspend point does not correctly captures the intent

**Section:** 11.4.4 [dcl.fct.def.coroutine] **Status:** Active **Submitter:** Gor Nishanov **Opened:** 2018-03-08 **Last modified:** 2018-05-05

*[EWG Approved Rapperswil-2018/06/09]*

#### Issue:

Common use of the `initial_suspend` point in asynchronous coroutines is to suspend the coroutine during the initial invocation and post a request to resume the coroutine by a different execution agent. If an execution agent at a later point cannot resume the coroutine, for example, because it is being shutdown, an error will be reported to a coroutine by resuming the coroutine and subsequently throwing an exception from `await_resume`.

Currently, the invocation of `initial_suspend` is specified in [dcl.fct.def.coroutine] as:

```
{
    P p;
    co_await p.initial_suspend(); // initial suspend point
    try { F } catch(...) { p.unhandled_exception(); }
    final_suspend:
    co_await p.final_suspend(); // final suspend point
}
```

As specified, an exception from `await_resume` of `initial_suspend` will be thrown outside of the try-catch and will not be captured by `p.unhandled_exception()` and whoever waits for eventual completion of a coroutine will never learn about its completion.

This is a specification defect. The intent is to capture an exception thrown by `await_resume` of an awaitable returned by `initial_suspend` within the try-catch enclosing of the user authored body `F`.

The correct behavior has been implemented in MSVC starting with version 2015 and in clang trunk.

### Proposed resolution:

Add underlying text to paragraph 11.4.4/3 as follows:

```
{
    P p;
    co_await p.initial_suspend(); // initial suspend point
    try {
        F
    } catch(...) { p.unhandled_exception(); }
    final_suspend:
    co_await p.final_suspend(); // final suspend point
}
```

except that any exception thrown after the *initial suspend point* and before the flow of execution reaches *F* also results in entering the *handler of the try-block and*, where an object denoted as *p* is the *promise object* of the coroutine and its type *P* is the *promise type* of the coroutine, ...

## 25. Allow unhandled exception escape the user-defined body of the coroutine and give it well defined semantics

**Section:** 11.4.4 [dcl.fct.def.coroutine] **Status:** Active **Submitter:** Eric Niebler **Opened:** 2018-03-10 **Last modified:** 2018-05-05

[EWG Approved Rapperswil-2018/06/09]

### Issue:

Currently, an unhandled exception can never escape the user-authored body of the coroutine with triggering undefined behavior.

```
{
    P p;
    co_await p.initial_suspend(); // initial suspend point
    try {
        F // user authored body
    } catch(...) { p.unhandled_exception(); }
    final_suspend:
    co_await p.final_suspend(); // final suspend point
}
```

An exception from `F` is captured by the try-catch and a customization point `unhandled_exception` is called, where, typically, an `exception_ptr` is created and propagated to the consumer awaiting on async task, or, in case of a generator, will be delivered to the user when they dereference the iterator.

Though the current behavior is perfectly reasonable for asynchronous scenarios, it is sub-optimal for synchronous generators. Capturing an exception, storing it in an `exception_ptr` and then rethrowing the

exception during, say, iterator's operator\* is a needless work if the desired behavior is to let the exception propagate to the caller whenever it asks for the next value.

**Background information:** When a coroutine is first invoked, any exception thrown before entering the user-authored body (for example allocation failure, promise constructor failure, failure to copy parameters, etc) propagates into the caller as with any normal function call. However, when the coroutine suspends and subsequently resumed, if an exception is thrown by an evaluation of `p.unhandled_exception()` or an evaluation of `co_await p.final_suspend()` the behavior is undefined. Note that a coroutine can be only resumed or destroyed when suspended at a particular suspend point. An exception leaving the coroutine at arbitrary point of the execution leaves the coroutine in the undefined state.

The proposed resolution is to eliminate the undefined behavior in the following manner:

1. Allow an exception to escape `p.unhandled_exception()` and, in that case, consider the coroutine to be at the final suspend point. Reminder: when a coroutine is at the final suspend point, the coroutine can only be destroyed and a call to member function `done()` of the coroutine handle associated with that coroutine returns `true`.
2. Eliminate possibility of an exception being thrown from evaluation of an expression `co_await p.final_suspend()` by stating that `final_suspend` member function of the coroutine promise and `await_resume`, `await_ready`, and `await_suspend` members of the object returned from `final_suspend` shall have non-throwing exception specification.

This resolution allows generator implementations to define `unhandled_exception` as follows:

```
void unhandled_exception() { throw; }
```

With this implementation, if a user of the generator pulls the next value, and during computation of the next value an exception will occur in the user authored body it will be propagate back to the user and the coroutine will be put into a final suspend state and ready to be destroyed when generator destructors is run.

### Proposed Wording:

In subclause 11.4.4, add two new paragraph after paragraph 11.

12. If the evaluation of an expression `p.unhandled_exception()` exits via an exception, the coroutine is considered suspended at the final suspend point.
13. The member functions `final_suspend`, `await_resume`, `await_ready`, `await_suspend` and operator `co_await` (if any) used in evaluation of expression `co_await p.final_suspend()` shall have non-throwing exception specification.

## 26. Relax requirements on a coroutine\_handle passed to await\_suspend

**Section:** 8.3.8 [expr.await] **Status:** Active **Submitter:** Gor Nishanov **Opened:** 2018-03-10 **Last modified:** 2018-03-10

[EWG Approved Rapperswil-2018/06/09]

### Issue:

One of the implementation strategies for coroutines is to chop original function into as many functions (parts) as there are suspend points. In that case, it is possible for a compiler create a unique per suspend per function coroutine\_handle which resume and destroy members can be direct calls to corresponding parts of the function.

Though no compiler is doing it now, we can allow implementors to experiment with this approach by relaxing the requirement on the coroutine\_handle passed to `await_suspend`.

### Proposed wording:

Add underlined text to 8.3.8/3.5:

(3.5) — h is an object of type convertible to `std::experimental::coroutine_handle<P>` referring to the enclosing coroutine.

## 27. Make suspension in dynamic initializers of static and `thread_local` local variables ill-formed

**Section:** 8.3.8/2 [expr.await] **Status:** Active **Submitter:** Richard Smith **Opened:** 2018-03-25 **Last modified:** 2018-03-25

[EWG Approved Rapperswil-2018/06/09]

### Proposed wording:

Add underlined text to 8.3.8/2:

An *await-expression* shall appear only in a potentially-evaluated expression within the *compound-statement* of a *function-body* outside of a *handler* (Clause 18). In a declaration-statement or in the simple-declaration (if any) of a for-init-statement, an await-expression shall appear only in an initializer of that declaration-statement or simple-declaration unless it is used to initialize a block-scope variable with static or thread storage duration.

---

## 28. Simplify stable name for Coroutines to be [def.coroutine]

**Section:** 11.4.4 [dcl.fct.def.coroutine] **Status:** Active **Submitter:** Gor Nishanov **Opened:** 2018-03-10 **Last modified:** 2018-03-10

### Proposed wording:

[~~dcl.fct~~.def.coroutine]

---

## Adopted at Rapperswil 2018 issues

## 29. Absence of `const` on `coroutine_handle::resume()` makes lambdas verbose

**Section:** 21.11.2 [coroutine.handle] **Status:** Active **Submitter:** (Many) **Opened:** 2018-03-08 **Last modified:** 2018-03-08

Approved by LEWG June 7, 2018 to reinstate `const` to `resume/destroy/operator()` of `coroutine_handle`

Adopted at Rapperswil 2018

### Issue:

During LEWG/LWG review of Coroutine TS, `const` on `coroutine_handle::resume()` was removed. While it does follow LEWG/LWG convention of only putting `const` on members that are safe to call concurrently from multiple threads, it does make writing lambdas capturing `coroutine_handle` more verbose.

**used to write:** [h]{ h.resume();}

**now have to write:** [h]\_(.) mutable{ h.resume();}

Customers complained and I have recorded this as an issue.

## Wording relative to N4736

Add underlined text to synopsis of 21.11.2/[coroutine.handle] as follows:

```
...
// 21.11.2.4 resumption
void operator>()() const;
void resume() const;
void destroy() const;
...
```

Add underlined text to subclause 21.11.2.4/[coroutine.handle.resumption] as follows:

```
...
void operator>()() const;
void resume() const;
...
void destroy() const;
```

---

## Not ready issues

---

### **30. Re-enable coroutine return type deduction when coroutine task and generator types are available**

**Section:** 10.1.7.4 [coroutine.handle] **Status:** Not ready **Submitter:** Gor Nishanov **Opened:** 2018-05-05 **Last modified:** 2018-05-05

#### **Issue:**

We stripped out automatic return type deduction for coroutines from N4499 in 2015. Put back the wording to do it once the appropriate types are available in the standard library.

### **31. Add a note warning about thread switching near await and/or coroutine\_handle wording.**

**Status:** Not ready **Submitter:** SG1 **Opened:** 2018-06-08

#### **Issue:**

Add a note warning about thread switching near await and/or coroutine\_handle wording

*Paper requested. Volunteer found*

---

### **32. Add a normative text making it UB to migrate coroutines between certain kind of execution agents.**

**Status:** Not ready **Submitter:** SG1 **Opened:** 2018-06-08

**Issue:**

Add a normative text making it UB to migrate coroutines between certain kind of execution agents. Clarify that migrating between `std::threads` is OK. But migrating between CPU and GPU is UB.

*Paper requested. Volunteer found*

---

**33. Parameter copy wording does not capture the intent.**

**Section:** 11.4.4/11 [dcl.fct.def.coroutine] **Status:** Not ready **Submitter:** Mathias Stearn **Opened:** 2018-06-09

**Issue:**

The intent is that copies/moves of parameters (if required) are created preserving the exact type (including references, r-references, etc). The wording in 11.4.4[dcl.fct.def.coroutine]/11 does not seem to express that clearly.

*Wording requested. Volunteer not found*